Deep Reinforcement Learning

Unit 5: Unity ML-Agents

Hugging Face Deep RL Course

Complete MDX-to-LaTeX study edition

Source: Hugging Face Deep RL Course (original English text).

Contents

1 An Introduction to Unity ML-Agents **2**

2 How do Unity ML-Agents work? **3**
- 2.1 What is Unity ML-Agents? . . . 3
- 2.2 The six components . . . 4
- 2.3 Inside the Learning Component . . . 4

3 The SnowballTarget Environment **6**
- 3.1 The agent's Goal . . . 6
- 3.2 The reward function and the reward engineering problem . . . 7
- 3.3 The observation space . . . 8
- 3.4 The action space . . . 9

4 The Pyramid environment **10**
- 4.1 The reward function . . . 10
- 4.2 The observation space . . . 11
- 4.3 The action space . . . 12

5 (Optional) What is Curiosity in Deep Reinforcement Learning? **13**
- 5.1 Two Major Problems in Modern RL . . . 13
- 5.2 So what is Curiosity? . . . 14
- 5.3 Hands-on . . . 15
- 5.4 Unit 5: An Introduction to ML-Agents . . . 17
- 5.5 Let's train our agents . . . 18

6 Quiz **27**
- 6.1 Quiz solutions and explanations . . . 28

7 Conclusion **29**

8 Bonus: Learn to create your own environments with Unity and MLAgents **30**
- 8.1 Step 1: Know how to use Unity . . . 30
- 8.2 Step 2: Create the simplest environment with this tutorial . . . 31
- 8.3 Step 3: Iterate and create nice environments . . . 31

1 An Introduction to Unity ML-Agents



Figure 1: Unity ML-Agents course unit cover [original figure]

One of the challenges in Reinforcement Learning is **creating environments**. Fortunately for us, we can use game engines to do so. These engines, such as Unity, Godot or Unreal Engine, are programs made to create video games. They are perfectly suited for creating environments: they provide physics systems, 2D/3D rendering, and more.

One of them, Unity, created the Unity ML-Agents Toolkit, a plugin based on the game engine Unity that allows us **to use the Unity Game Engine as an environment builder to train agents**. In the first bonus unit, this is what we used to train Huggy to catch a stick!

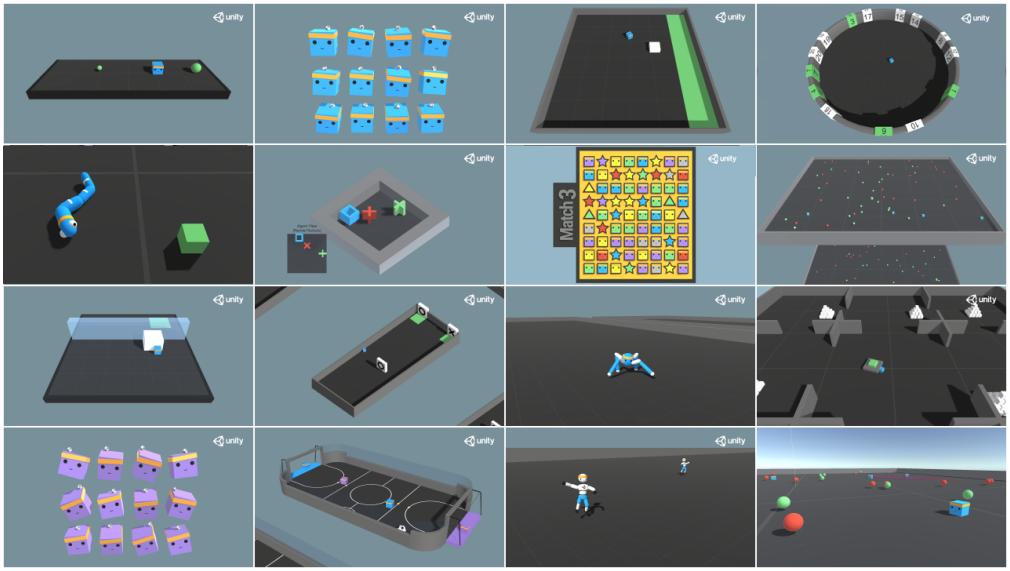


Figure 2: Source: ML-Agents documentation (source: https://github.com/Unity-Technologies/ml-agents) [original figure]

Unity ML-Agents Toolkit provides many exceptional pre-made environments, from playing football (soccer), learning to walk, and jumping over big walls.

In this Unit, we'll learn to use ML-Agents, but **don't worry if you don't know how to use the Unity Game Engine**: you don't need to use it to train your agents.

So, today, we're going to train two agents:

- The first one will learn to **shoot snowballs onto a spawning target**.
- The second needs to **press a button to spawn a pyramid, then navigate to the pyramid, knock it over, and move to the gold brick at the top**. To do that, it will need to explore its environment, which will be done using a technique called curiosity.

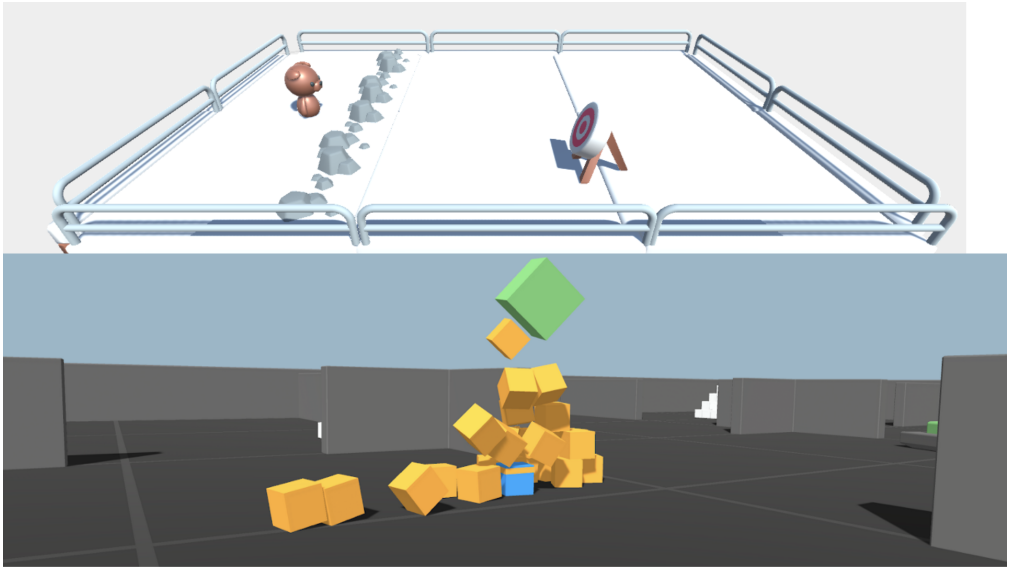


Figure 3: SnowballTarget and Pyramids learning environments [original figure]

Then, after training, **you'll push the trained agents to the Hugging Face Hub**, and you'll be able to **visualize them playing directly on your browser without having to use the Unity Editor**.

Doing this Unit will **prepare you for the next challenge: AI vs. AI where you will train agents in multi-agents environments and compete against your classmates' agents**.

Sound exciting? Let's get started!

2 How do Unity ML-Agents work?

Before training our agent, we need to understand **what ML-Agents is and how it works**.

2.1 What is Unity ML-Agents?

Unity ML-Agents is a toolkit for the game engine Unity that **allows us to create environments using Unity or use pre-made environments to train our agents**.

It's developed by Unity Technologies, the developers of Unity, one of the most famous Game Engines used by the creators of Firewatch, Cuphead, and Cities: Skylines.



Figure 4: Firewatch was made with Unity [original figure]

2.2 The six components

With Unity ML-Agents, you have six essential components:

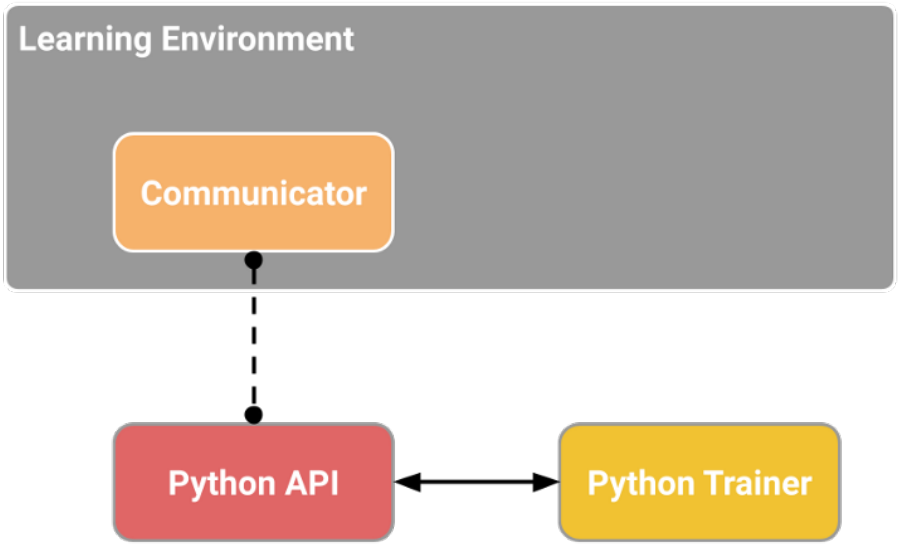


Figure 5: Source: Unity ML-Agents Documentation (source: https://unity-technologies.github.io/ml-agents/) [original figure]

- The first is the *Learning Environment*, which contains **the Unity scene (the environment) and the environment elements** (game characters).
- The second is the *Python Low-level API*, which contains **the low-level Python interface for interacting and manipulating the environment**. It's the API we use to launch the training.
- Then, we have the *External Communicator* that **connects the Learning Environment (made with C#) with the low level Python API (Python)**.
- The *Python trainers*: the **Reinforcement algorithms made with PyTorch (PPO, SAC. . .)**.
- The *Gym wrapper*: to encapsulate the RL environment in a gym wrapper.
- The *PettingZoo wrapper*: PettingZoo is the multi-agents version of the gym wrapper.

2.3 Inside the Learning Component

Inside the Learning Component, we have **two important elements**:

- The first is the *agent component*, the actor of the scene. We'll **train the agent by optimizing its policy** (which will tell us what action to take in each state). The policy is called the *Brain*.
- Finally, there is the *Academy*. This component **orchestrates agents and their decision-making processes**. Think of this Academy as a teacher who handles Python API requests.

To better understand its role, let's remember the RL process. This can be modeled as a loop that works like this:

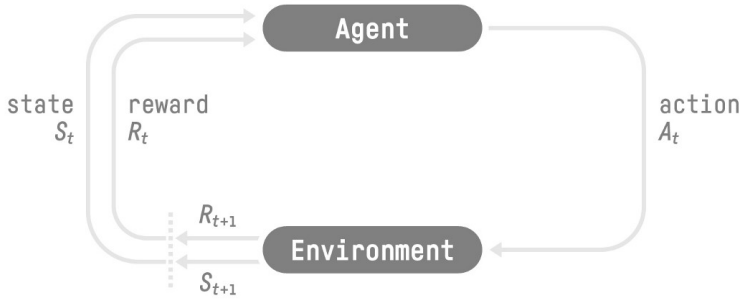


Figure 6: The RL Process: a loop of state, action, reward and next state; Source: Reinforcement Learning: An Introduction, Richard Sutton and Andrew G. Barto (source: http://incompleteideas.net/book/RLbook2020.pdf) [original figure]

Now, let's imagine an agent learning to play a platform game. The RL process looks like this:

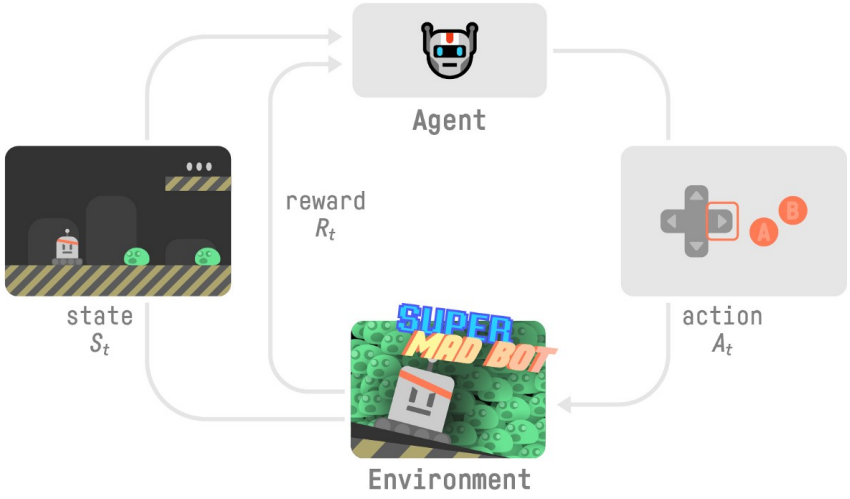


Figure 7: Reinforcement learning process illustrated in a game [original figure]

- Our Agent receives **state** S_0 from the **Environment** — we receive the first frame of our game (Environment).
- Based on that **state** S_0**,** the Agent takes **action** A_0 — our Agent will move to the right.
- The environment goes to a **new state** S_1 — new frame.
- The environment gives some **reward** R_1 to the Agent — we're not dead *(Positive Reward +1)*.

This RL loop outputs a sequence of **state, action, reward and next state.** The goal of the agent is to **maximize the expected cumulative reward**.

The Academy will be the one that will **send the order to our Agents and ensure that agents are in sync**:

- Collect Observations
- Select your action using your policy
- Take the Action
- Reset if you reached the max step or if you're done.



Figure 8: Unity ML-Agents Academy and the decision loop [original figure]

Now that we understand how ML-Agents works, **we're ready to train our agents.**

3 The SnowballTarget Environment

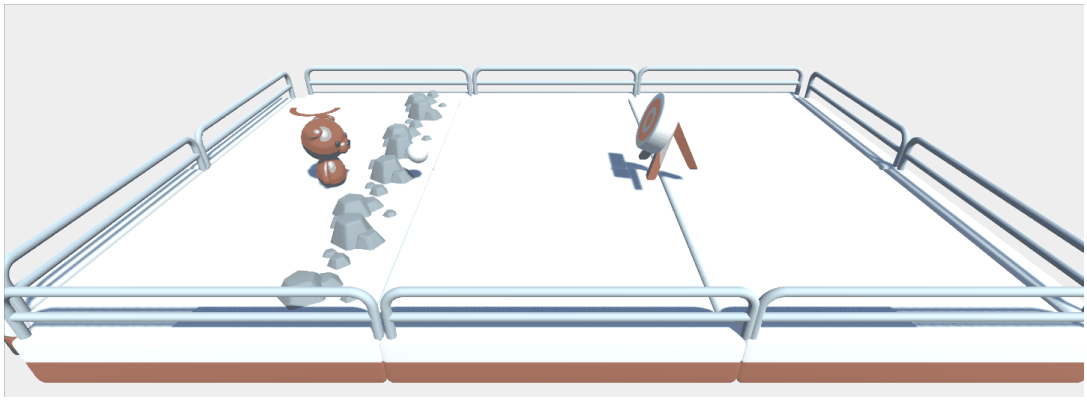


Figure 9: SnowballTarget learning environment (animation still) [original animation]

SnowballTarget is an environment we created at Hugging Face using assets from Kay Lousberg. We have an optional section at the end of this Unit **if you want to learn to use Unity and create your environments**.

3.1 The agent's Goal

The first agent you're going to train is called Julien the bear . Julien is trained **to hit targets with snowballs**.

The Goal in this environment is that Julien **hits as many targets as possible in the limited time** (1000 timesteps). It will need **to place itself correctly in relation to the target and shoot**to do that.

In addition, to avoid "snowball spamming" (aka shooting a snowball every timestep), **Julien has a "cool off" system** (it needs to wait 0.5 seconds after a shoot to be able to shoot again).

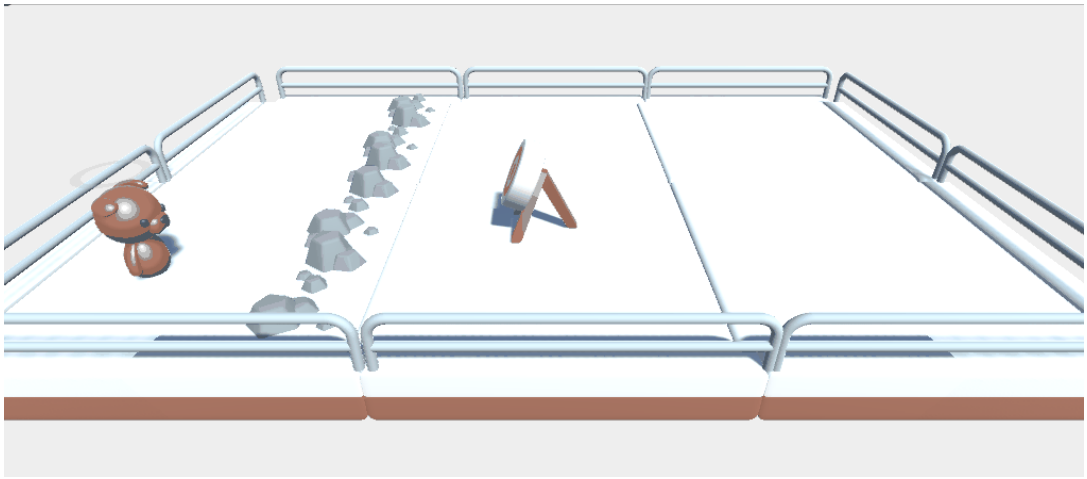


Figure 10: The agent needs to wait 0.5s before being able to shoot a snowball again [original animation]

3.2 The reward function and the reward engineering problem

The reward function is simple. **The environment gives a +1 reward every time the agent's snowball hits a target**. Because the agent's Goal is to maximize the expected cumulative reward, **it will try to hit as many targets as possible**.

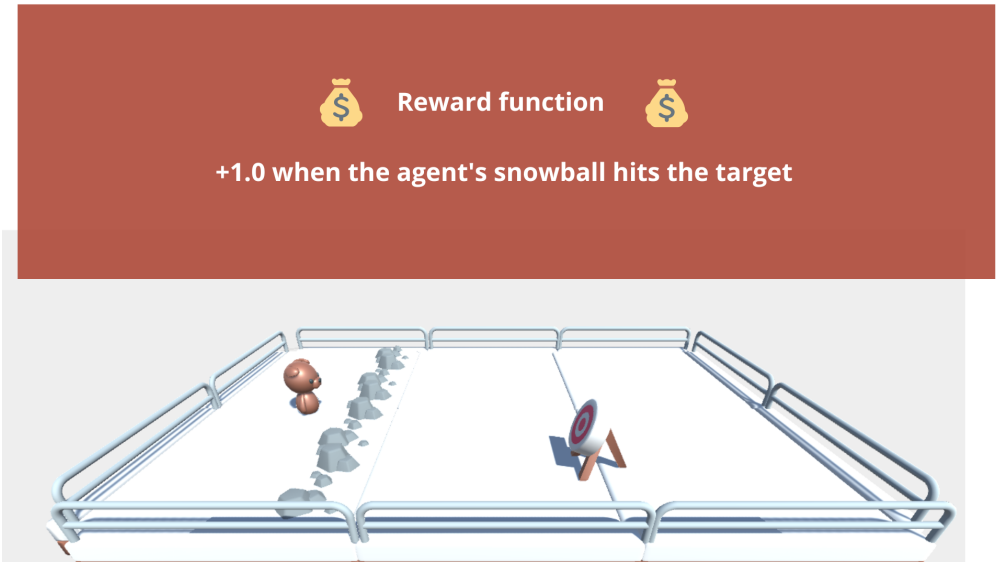


Figure 11: SnowballTarget reward mechanism [original figure]

We could have a more complex reward function (with a penalty to push the agent to go faster, for example). But when you design an environment, you need to avoid the *reward engineering problem*, which is having a too complex reward function to force your agent to behave as you want it to do. Why? Because by doing that, **you might miss interesting strategies that the agent will find with a simpler reward function**.

In terms of code, it looks like this:

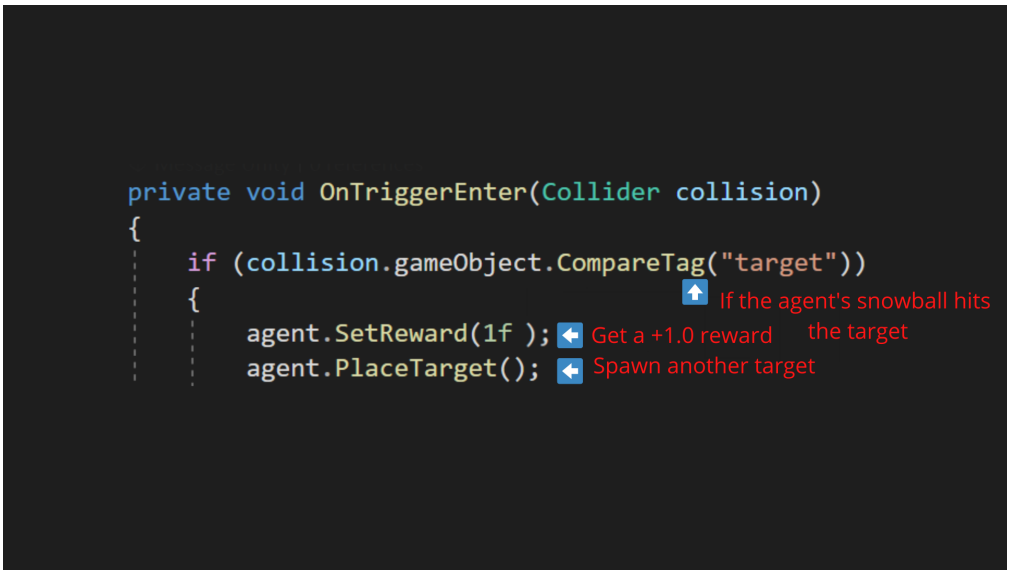


Figure 12: SnowballTarget reward computation in code [original figure]

3.3 The observation space

Regarding observations, we don't use normal vision (frame), but **we use raycasts**.

Think of raycasts as lasers that will detect if they pass through an object.

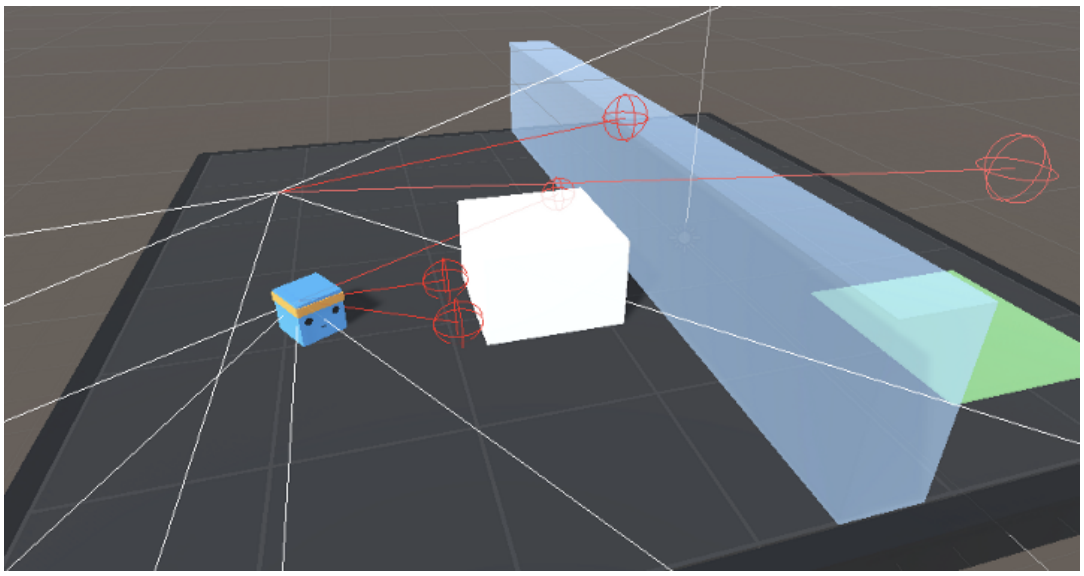


Figure 13: Source: ML-Agents documentation (source: https://github.com/Unity-Technologies/ml-agents) [original figure]

In this environment, our agent has multiple set of raycasts:

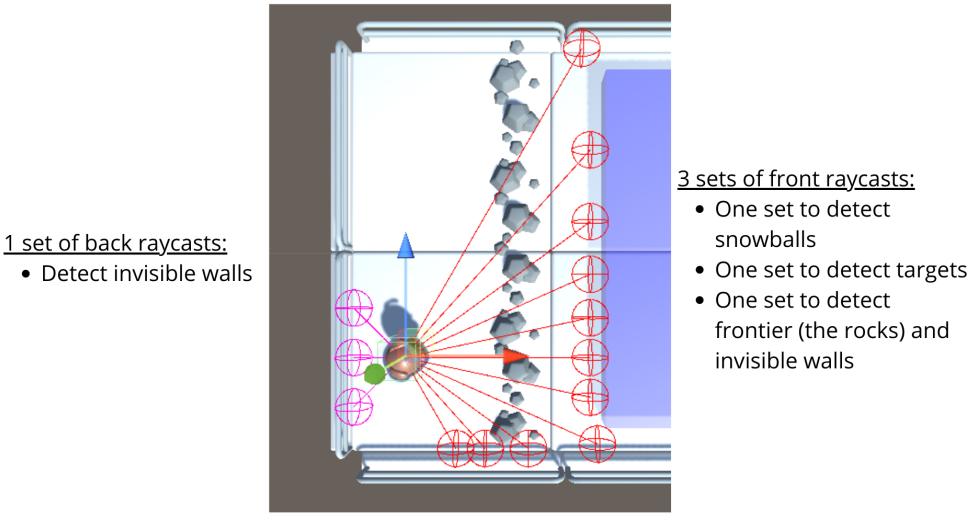


Figure 14: SnowballTarget agent raycast observations [original figure]

In addition to raycasts, the agent gets a "can I shoot" bool as observation.

```
/// <summary>
/// Collect the observations (vector obs)
/// </summary>
/// <param name="sensor"></param>
8 références
public override void CollectObservations(VectorSensor sensor)
{
    // Can I shoot or not? (Bool)
    sensor.AddObservation(possibleShoot);
}
```

Figure 15: SnowballTarget observation collection code [original figure]

3.4 The action space

The action space is discrete:

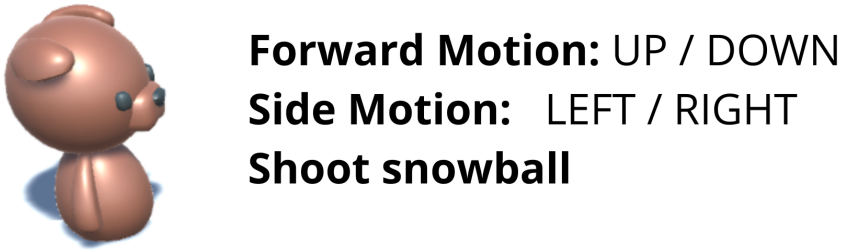


Figure 16: SnowballTarget discrete action space [original figure]

4 The Pyramid environment

The goal in this environment is to train our agent to **get the gold brick on the top of the Pyramid. To do that, it needs to press a button to spawn a Pyramid, navigate to the Pyramid, knock it over, and move to the gold brick at the top**.

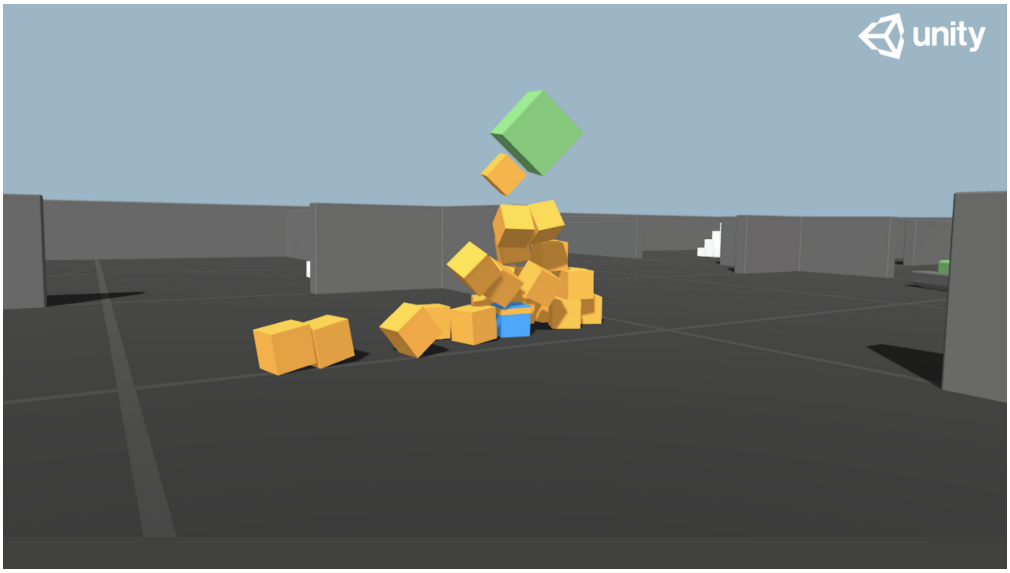


Figure 17: The Pyramids learning environment [original figure]

4.1 The reward function

The reward function is:

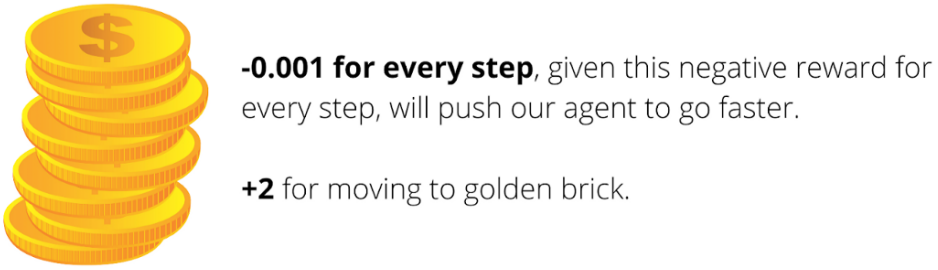


Figure 18: Reward structure of the Pyramids environment [original figure]

In terms of code, it looks like this

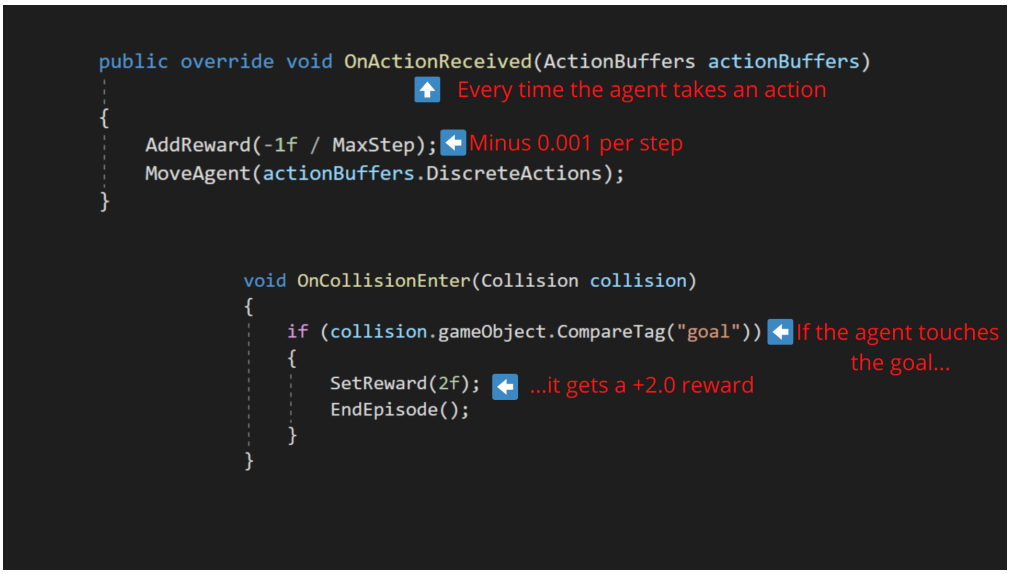


Figure 19: Pyramids reward computation in code [original figure]

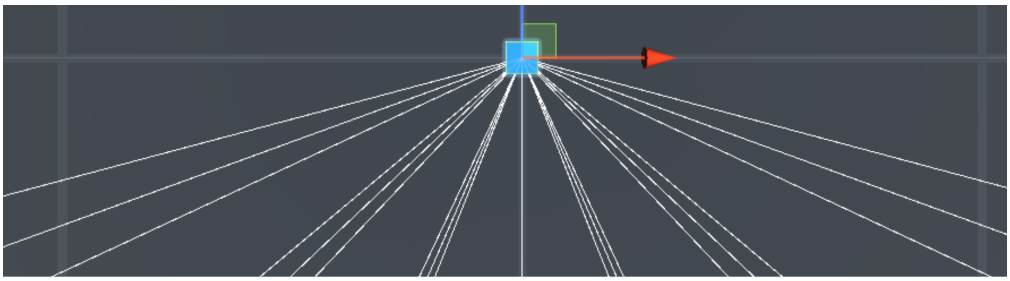
To train this new agent that seeks that button and then the Pyramid to destroy, we'll use a combination of two types of rewards:

- The *extrinsic one* given by the environment (illustration above).
- But also an *intrinsic* one called **curiosity**. This second will **push our agent to be curious, or in other terms, to better explore its environment**.

If you want to know more about curiosity, the next section (optional) will explain the basics.

4.2 The observation space

In terms of observation, we **use 148 raycasts that can each detect objects** (switch, bricks, golden brick, and walls.)



3 sets of front raycasts:

- All three detect:
  - Blocks
  - Walls
  - Goal (green cube)
  - Switch on
  - Switch off
  - Stone

Figure 20: Pyramids agent raycast-based observations [original figure]

We also use a **boolean variable indicating the switch state** (did we turn on or off the switch to spawn the Pyramid) and a vector that **contains the agent's speed**.



Figure 21: Pyramids observation collection code [original figure]

4.3 The action space

The action space is **discrete** with four possible actions:

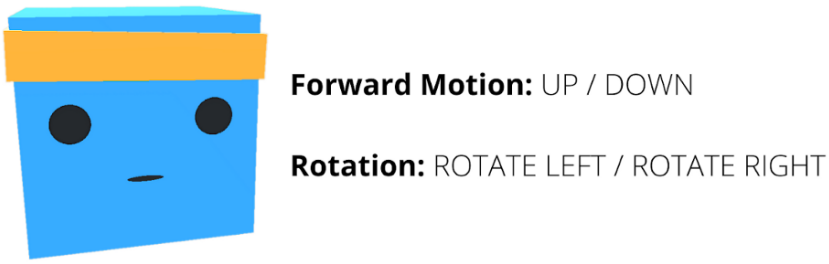


Figure 22: Pyramids discrete action space [original figure]

5 (Optional) What is Curiosity in Deep Reinforcement Learning?

This is an (optional) introduction to Curiosity. If you want to learn more, you can read two additional articles where we dive into the mathematical details:

- Curiosity-Driven Learning through Next State Prediction
- Random Network Distillation: a new take on Curiosity-Driven Learning

5.1 Two Major Problems in Modern RL

To understand what Curiosity is, we first need to understand the two major problems with RL:

First, the *sparse rewards problem:* that is, **most rewards do not contain information, and hence are set to zero**.

Remember that RL is based on the *reward hypothesis*, which is the idea that each goal can be described as the maximization of the rewards. Therefore, rewards act as feedback for RL agents; **if they don't receive any, their knowledge of which action is appropriate (or not) cannot change**.



Figure 23: Source: Thanks to the reward, our agent knows that this action at that state was good [original figure]

For instance, in Vizdoom, a set of environments based on the game Doom "DoomMyWayHome,"

your agent is only rewarded **if it finds the vest**. However, the vest is far away from your starting point, so most of your rewards will be zero. Therefore, if our agent does not receive useful feedback (dense rewards), it will take much longer to learn an optimal policy, and **it can spend time turning around without finding the goal**.

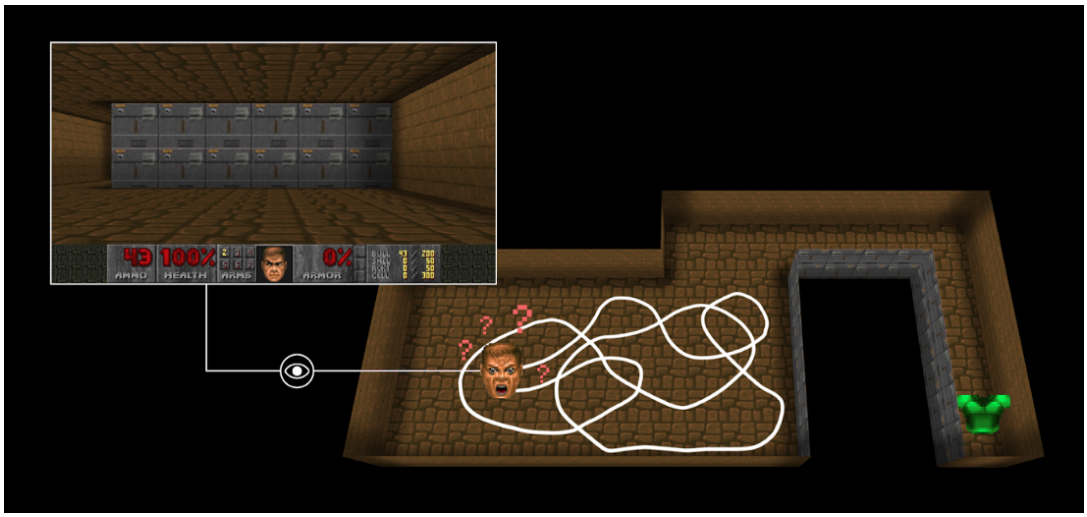


Figure 24: Sparse rewards and the DoomMyWayHome environment [original figure]

The second big problem is that **the extrinsic reward function is handmade; in each environment, a human has to implement a reward function**. But how we can scale that in big and complex environments?

5.2 So what is Curiosity?

A solution to these problems is **to develop a reward function intrinsic to the agent, i.e., generated by the agent itself**. The agent will act as a self-learner since it will be the student and its own feedback master.

This intrinsic reward mechanism is known as Curiosity because this reward pushes the agent to explore states that are novel/unfamiliar. To achieve that, our agent will receive a high reward when exploring new trajectories.

This reward is inspired by how humans act. **We naturally have an intrinsic desire to explore environments and discover new things**.

There are different ways to calculate this intrinsic reward. The classical approach (Curiosity through next-state prediction) is to calculate Curiosity **as the error of our agent in predicting the next state, given the current state and action taken**.

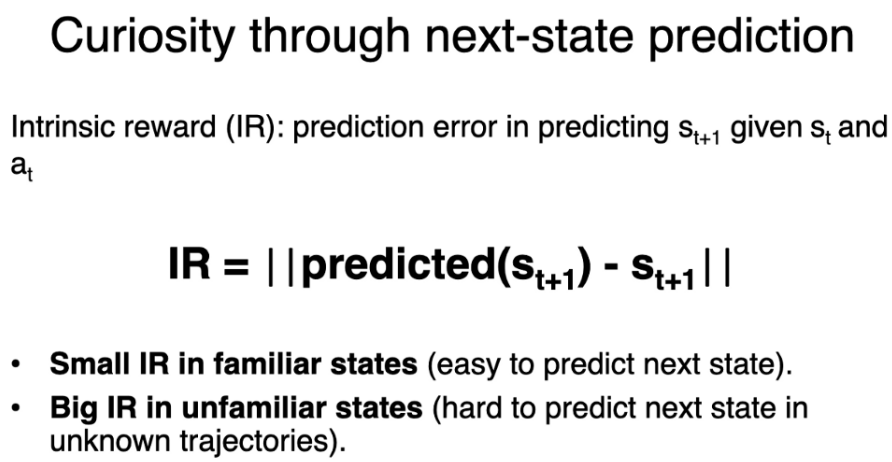


Figure 25: Intrinsic curiosity reward from next-state prediction error [original figure]

Because the idea of Curiosity is to **encourage our agent to perform actions that reduce the uncertainty in the agent's ability to predict the consequences of its actions** (uncertainty will be higher in areas where the agent has spent less time or in areas with complex dynamics).

If the agent spends a lot of time on these states, it will be good at predicting the next state (low Curiosity). On the other hand, if it's in a new, unexplored state, it will be hard to predict the following state (high Curiosity).

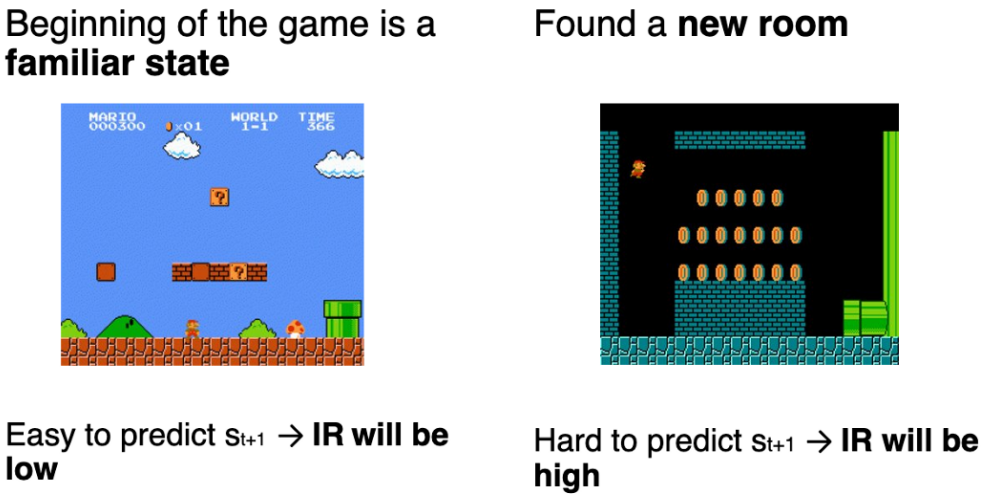


Figure 26: Curiosity and exploration in unfamiliar states [original figure]

Using Curiosity will push our agent to favor transitions with high prediction error (which will be higher in areas where the agent has spent less time, or in areas with complex dynamics) and **consequently better explore our environment**.

There's also **other curiosity calculation methods**. ML-Agents uses a more advanced one called Curiosity through random network distillation. This is out of the scope of the tutorial but if you're interested I wrote an article explaining it in detail.

5.3 Hands-on

Official Unit 5 practical notebook (Google Colab)

Community help

We learned what ML-Agents is and how it works. We also studied the two environments we're going to use. Now we're ready to train our agents!

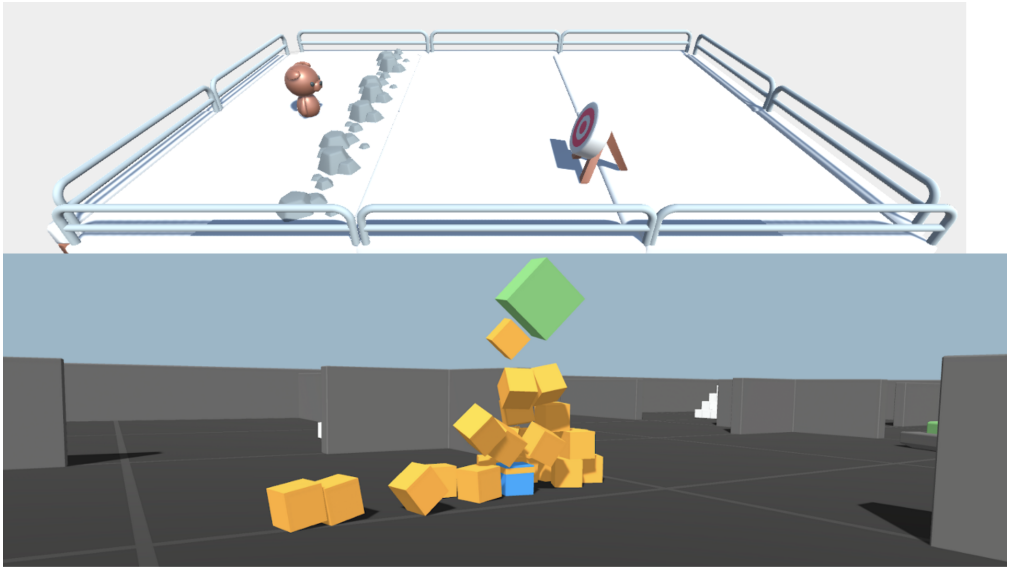


Figure 27: SnowballTarget and Pyramids learning environments [original figure]

To validate this hands-on for the certification process, you **just need to push your trained models to the Hub.** There are **no minimum results to attain** in order to validate this Hands On. But if you want to get nice results, you can try to reach the following:

- For Pyramids: Mean Reward = 1.75
- For SnowballTarget: Mean Reward = 15 or 30 targets shoot in an episode.

For more information about the certification process, check this section https://huggingface.co/deep-rl-course/en/unit0/introduction#certification-process

To start the hands-on, click on Open In Colab button :

Open in Google Colab

We strongly **recommend students use Google Colab for the hands-on exercises** instead of running them on their personal computers.

By using Google Colab, **you can focus on learning and experimenting without worrying about the technical aspects** of setting up your environments.

5.4 Unit 5: An Introduction to ML-Agents



Figure 28: Unity ML-Agents course unit cover [original figure]

In this notebook, you'll learn about ML-Agents and train two agents.

- The first one will learn to **shoot snowballs onto spawning targets**.
- The second needs to press a button to spawn a pyramid, then navigate to the pyramid, knock it over, **and move to the gold brick at the top**. To do that, it will need to explore its environment, and we will use a technique called curiosity.

After that, you'll be able **to watch your agents playing directly on your browser**.

For more information about the certification process, check this section https://huggingface.co/deep-rl-course/en/unit0/introduction#certification-process

Here is an example of what **you will achieve at the end of this unit.**

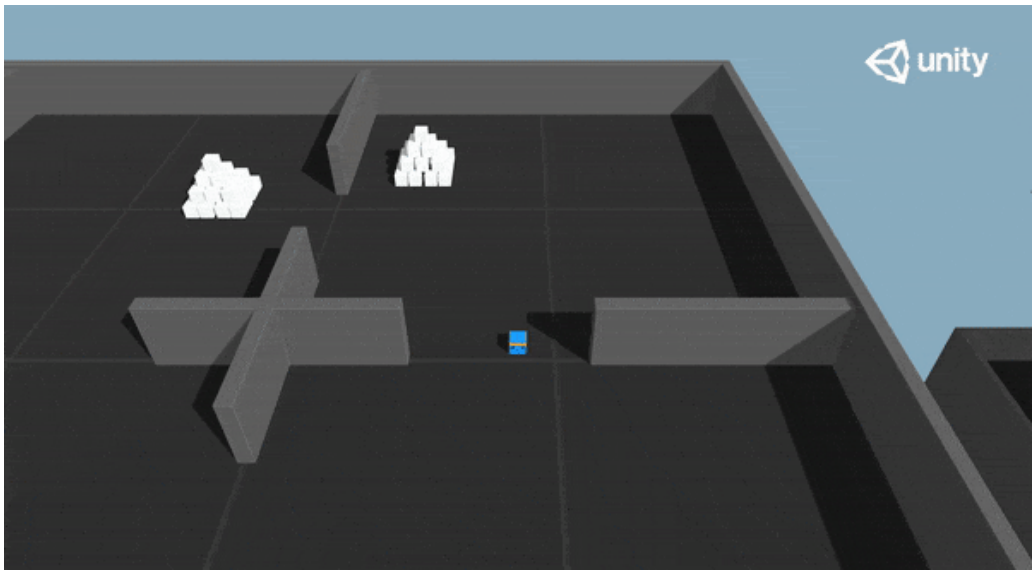


Figure 29: Pyramids agent demonstration (animation still) [original animation]

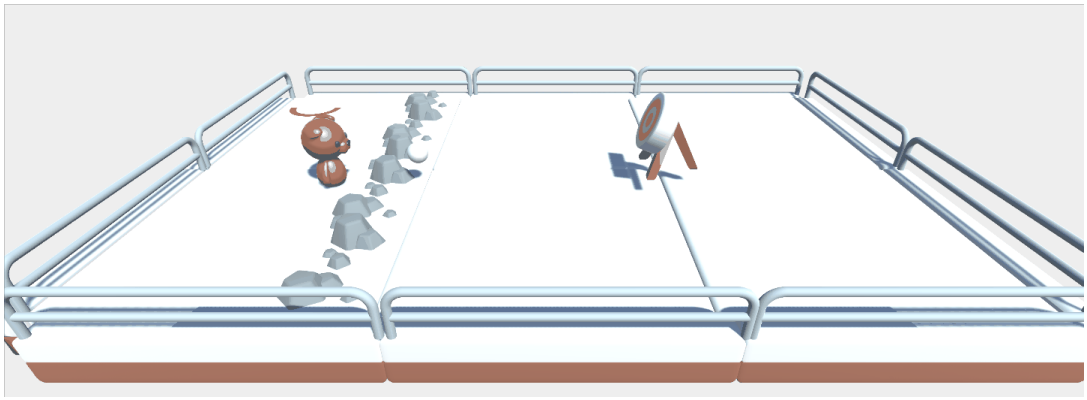


Figure 30: SnowballTarget learning environment (animation still) [original animation]

Environments:

- Pyramids
- SnowballTarget

RL-Library:

- ML-Agents

We're constantly trying to improve our tutorials, so **if you find some issues in this notebook**, please open an issue on the GitHub Repo.

Objectives of this notebook

At the end of the notebook, you will:

- Understand how **ML-Agents** works and the environment library.
- Be able to **train agents in Unity Environments**.

Prerequisites

Before diving into the notebook, you need to:

Study what ML-Agents is and how it works by reading Unit 5

5.5 Let's train our agents

Set the GPU

- To **accelerate the agent's training, we'll use a GPU**. To do that, go to `Runtime > Change Runtime type`

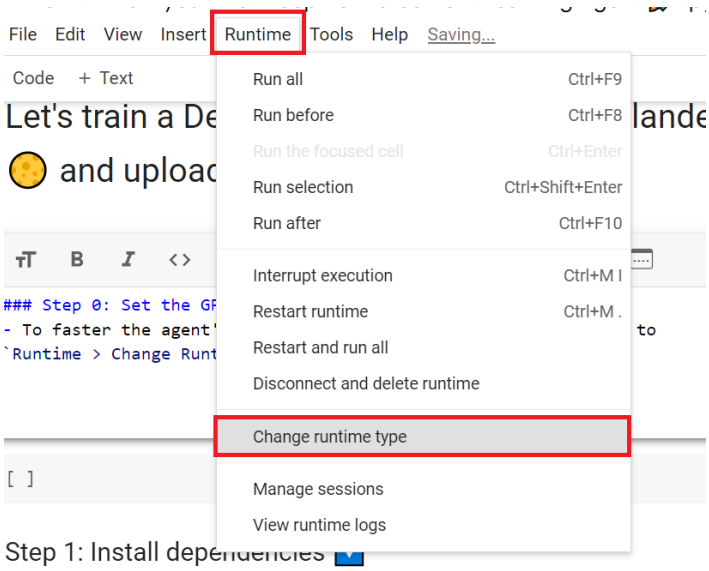


Figure 31: Selecting the Google Colab runtime type [original figure]

- `Hardware Accelerator > GPU`

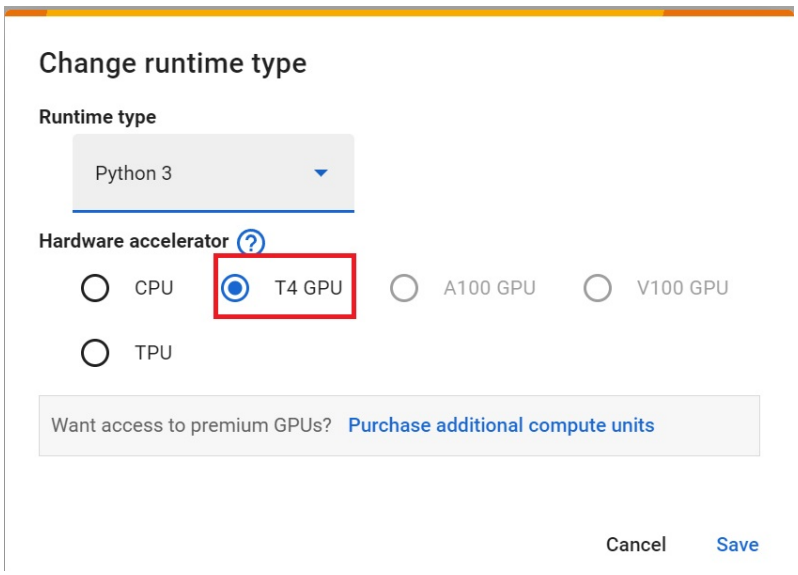


Figure 32: Enabling the GPU accelerator in Google Colab [original figure]

Clone the repository

- We need to clone the repository, that contains **ML-Agents.**

```
# Clone the repository (can take 3min)
git clone --depth 1 https://github.com/Unity-Technologies/ml-agents
```

Setup the Virtual Environment

- In order for the **ML-Agents** to run successfully in Colab, Colab's Python version must meet the library's Python requirements.
- We can check for the supported Python version under the `python_requires` parameter in the `setup.py` files. These files are required to set up the **ML-Agents** library for use and can be found in the following locations:
  - `/content/ml-agents/ml-agents/setup.py`

  - `/content/ml-agents/ml-agents-envs/setup.py`

- Colab's Current Python version(can be checked using `!python --version`) doesn't match the library's `python_requires` parameter, as a result installation may silently fail and lead to errors like these, when executing the same commands later:

  - `/bin/bash: line 1: mlagents-learn: command not found`
  - `/bin/bash: line 1: mlagents-push-to-hf: command not found`

- To resolve this, we'll create a virtual environment with a Python version compatible with the **ML-Agents** library.

`Note:` *For future compatibility, always check the* ***`python_requires`*** *parameter in the installation files and set your virtual environment to the maximum supported Python version in the given below script if the Colab's Python version is not compatible*

```
# Colab's Current Python Version (Incompatible with ML-Agents)
!python --version
```

```
# Install virtualenv and create a virtual environment
!pip install virtualenv
!virtualenv myenv

# Download and install Miniconda
!wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
!chmod +x Miniconda3-latest-Linux-x86_64.sh
!./Miniconda3-latest-Linux-x86_64.sh -b -f -p /usr/local

# Activate Miniconda and install Python ver 3.10.12
!source /usr/local/bin/activate
!conda install -q -y --prefix /usr/local python=3.10.12 ujson  # Specify the version here

# Set environment variables for Python and conda paths
!export PYTHONPATH=/usr/local/lib/python3.10/site-packages/
!export CONDA_PREFIX=/usr/local/envs/myenv
```

```
# Python Version in New Virtual Environment (Compatible with ML-Agents)
!python --version
```

Installing the dependencies

```
# Go inside the repository and install the package (can take 3min)
%cd ml-agents
pip3 install -e ./ml-agents-envs
pip3 install -e ./ml-agents
```

SnowballTarget

If you need a refresher on how this environment works check this section https://huggingface.co/deep-rl-course/unit5/snowball-target

Download and move the environment zip file in `./training-envs-executables/linux/`

- Our environment executable is in a zip file.
- We need to download it and place it to `./training-envs-executables/linux/`

- We use a linux executable because we use colab, and colab machines OS is Ubuntu (linux)

```
# Here, we create training-envs-executables and linux
mkdir ./training-envs-executables
mkdir ./training-envs-executables/linux
```

We downloaded the file SnowballTarget.zip from https://github.com/huggingface/Snowball-Target using `wget`

```
wget "https://github.com/huggingface/Snowball-Target/raw/main/SnowballTarget.zip" -O ./training-
    envs-executables/linux/SnowballTarget.zip
```

We unzip the executable.zip file

```
unzip -d ./training-envs-executables/linux/ ./training-envs-executables/linux/SnowballTarget.zip
```

Make sure your file is accessible

```
chmod -R 755 ./training-envs-executables/linux/SnowballTarget
```

Define the SnowballTarget config file

- In ML-Agents, you define the **training hyperparameters in config.yaml files.**

There are multiple hyperparameters. To understand them better, you should read the explanation for each one in the documentation

You need to create a `SnowballTarget.yaml` config file in ./content/ml-agents/config/ppo/

We'll give you a preliminary version of this config (to copy and paste into your `SnowballTarget.yaml file`), **but you should modify it**.

```
behaviors:
  SnowballTarget:
    trainer_type: ppo
    summary_freq: 10000
    keep_checkpoints: 10
    checkpoint_interval: 50000
    max_steps: 200000
    time_horizon: 64
    threaded: true
    hyperparameters:
      learning_rate: 0.0003
      learning_rate_schedule: linear
      batch_size: 128
      buffer_size: 2048
      beta: 0.005
      epsilon: 0.2
      lambd: 0.95
      num_epoch: 3
    network_settings:
      normalize: false
      hidden_units: 256
      num_layers: 2
      vis_encode_type: simple
    reward_signals:
      extrinsic:
        gamma: 0.99
        strength: 1.0
```

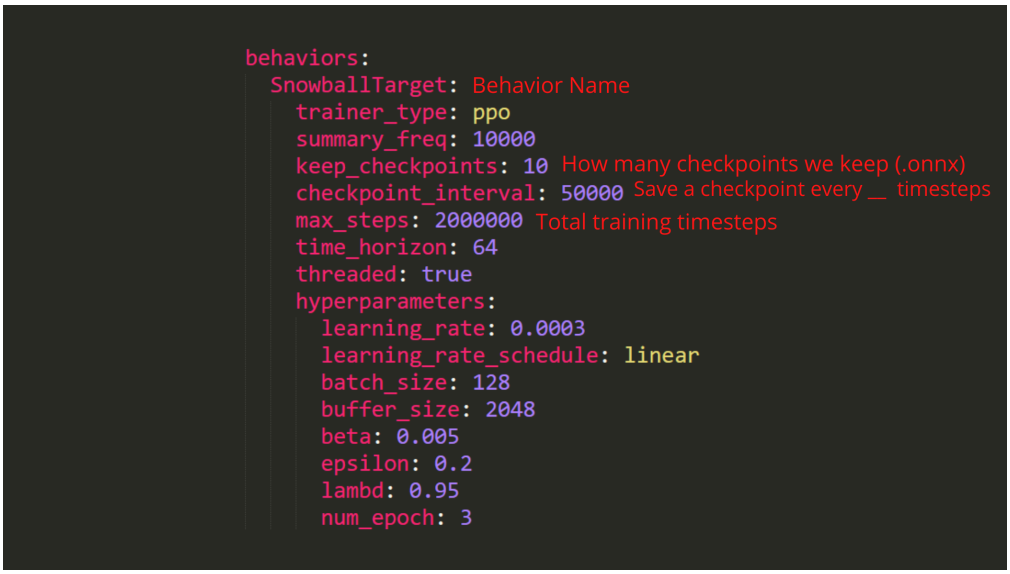


Figure 33: SnowballTarget PPO training configuration, part one [original figure]



Figure 34: SnowballTarget PPO training configuration, part two [original figure]

As an experiment, try to modify some other hyperparameters. Unity provides very good documentation explaining each of them here.

Now that you've created the config file and understand what most hyperparameters do, we're ready to train our agent .

Train the agent To train our agent, we need to **launch mlagents-learn and select the executable containing the environment.**

We define four parameters:

1. `mlagents-learn <config>`: the path where the hyperparameter config file is.
2. `--env`: where the environment executable is.
3. `--run_id`: the name you want to give to your training run id.
4. `--no-graphics`: to not launch the visualization during the training.

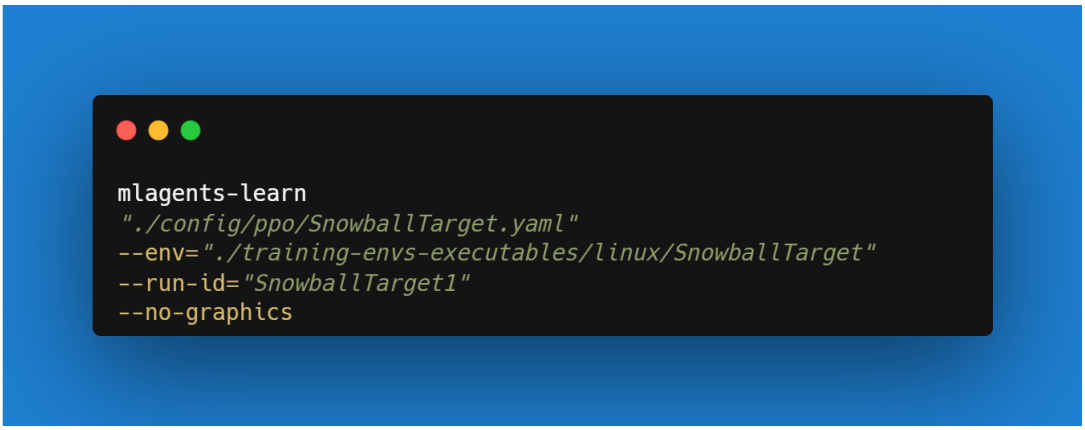


Figure 35: ML-Agents command-line training configuration [original figure]

Train the model and use the `--resume` flag to continue training in case of interruption.

> It will fail the first time if and when you use `--resume`. Try rerunning the block to bypass the error.

The training will take 10 to 35min depending on your config. Go take a coffee you deserve it .

```
mlagents-learn ./config/ppo/SnowballTarget.yaml --env=./training-envs-executables/linux/
    SnowballTarget/SnowballTarget --run-id="SnowballTarget1" --no-graphics
```

Push the agent to the Hugging Face Hub

- Now that we've trained our agent, we're **ready to push it to the Hub and visualize it playing on your browser.**

To be able to share your model with the community, there are three more steps to follow:

1. (If it's not already done) create an account to HF -> https://huggingface.co/join
2. Sign in and store your authentication token from the Hugging Face website.

- Create a new token (https://huggingface.co/settings/tokens) **with write role**

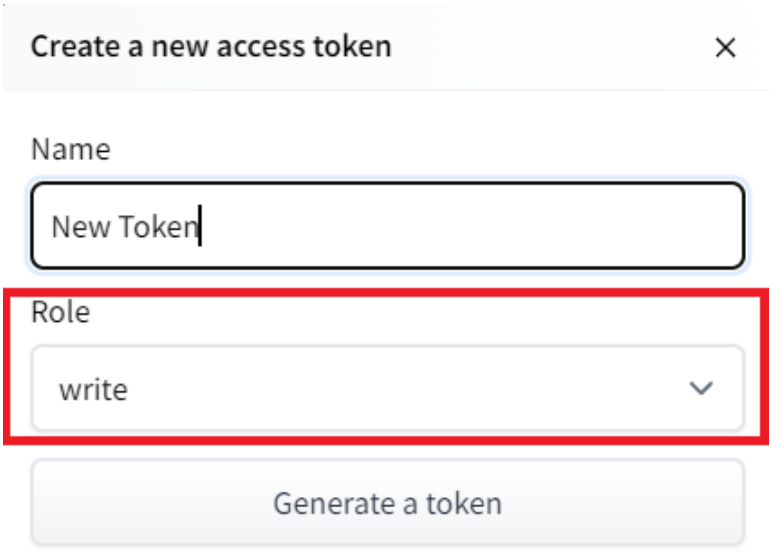


Figure 36: Creating a Hugging Face access token [original figure]

- Copy the token
- Run the cell below and paste the token

```
from huggingface_hub import notebook_login

notebook_login()
```

If you don't want to use Google Colab or a Jupyter Notebook, you need to use this command instead: `huggingface-cli login`

Then we need to run `mlagents-push-to-hf`.

And we define four parameters:

1. `--run-id`: the name of the training run id.
2. `--local-dir`: where the agent was saved, it's results/, so in my case results/First Training.
3. `--repo-id`: the name of the Hugging Face repo you want to create or update. It's always `<your huggingface username>/<the repo name>` If the repo does not exist **it will be created automatically**
4. `--commit-message`: since HF repos are git repositories you need to give a commit message.

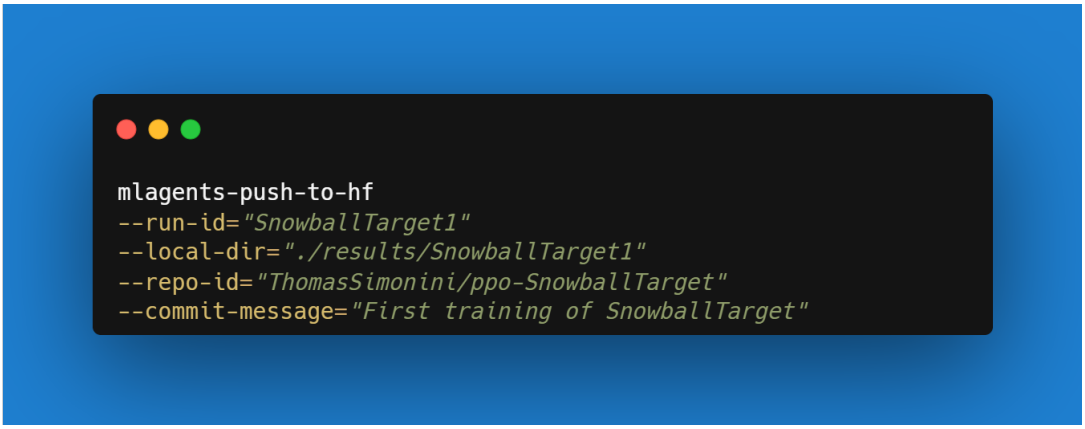


Figure 37: ML-Agents model publishing command [original figure]

For instance:

```
mlagents-push-to-hf  --run-id="SnowballTarget1" --local-dir="./results/SnowballTarget1" --repo-id="
    ThomasSimonini/ppo-SnowballTarget"  --commit-message="First Push"
```

```
mlagents-push-to-hf  --run-id= # Add your run id  --local-dir= # Your local dir  --repo-id= # Your
    repo id  --commit-message= # Your commit message
```

If everything worked you should see this at the end of the process (but with a different url) :

```
Your model is pushed to the hub. You can view your model here: https://huggingface.co/
    ThomasSimonini/ppo-SnowballTarget
```

It's the link to your model. It contains a model card that explains how to use it, your Tensorboard, and your config file. **What's awesome is that it's a git repository, which means you can have different commits, update your repository with a new push, etc.**

But now comes the best: **being able to visualize your agent online .**

Watch your agent playing This step it's simple:

1. Go here: https://huggingface.co/spaces/ThomasSimonini/ML-Agents-SnowballTarget
2. Launch the game and put it in full screen by clicking on the bottom right button

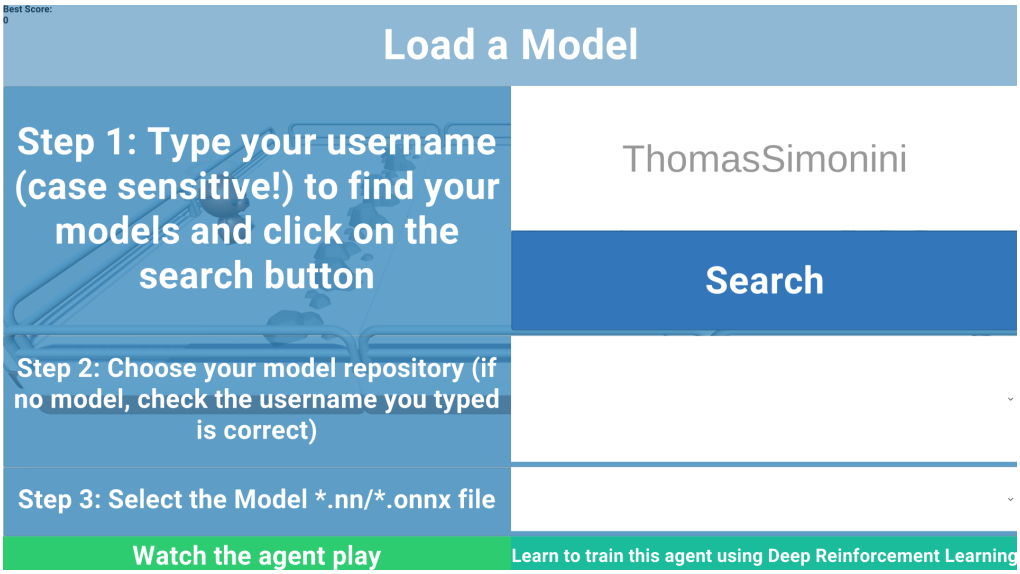


Figure 38: Loading a trained SnowballTarget agent in a Hugging Face Space [original figure]

1. In step 1, type your username (your username is case sensitive: for instance, my username is ThomasSimonini not thomassimonini or ThOmasImoNInI) and click on the search button.
2. In step 2, select your model repository.
3. In step 3, **choose which model you want to replay**:

- I have multiple ones, since we saved a model every 500000 timesteps.
- But since I want the most recent one, I choose `SnowballTarget.onnx`

It's good **to try with different models steps to see the improvement of the agent.**

And don't hesitate to share the best score your agent gets on discord in the #rl-i-made-this channel

Now let's try a more challenging environment called Pyramids.

Pyramids

Download and move the environment zip file in `./training-envs-executables/linux/`

- Our environment executable is in a zip file.
- We need to download it and place it into `./training-envs-executables/linux/`
- We use a linux executable because we're using colab, and the colab machine's OS is Ubuntu (linux)

We downloaded the file Pyramids.zip from from https : / / huggingface . co / spaces / unity / ML-Agents-Pyramids/resolve/main/Pyramids.zip using `wget`

```
wget "https://huggingface.co/spaces/unity/ML-Agents-Pyramids/resolve/main/Pyramids.zip" -O ./
    training-envs-executables/linux/Pyramids.zip
```

Unzip it

```
unzip -d ./training-envs-executables/linux/ ./training-envs-executables/linux/Pyramids.zip
```

Make sure your file is accessible

```
chmod -R 755 ./training-envs-executables/linux/Pyramids/Pyramids
```

Modify the PyramidsRND config file

- Contrary to the first environment, which was a custom one, **Pyramids was made by the Unity team**.
- So the PyramidsRND config file already exists and is in `./content/ml-agents/config/ppo/PyramidsRND.yaml`
- You might ask why "RND" is in PyramidsRND. RND stands for *random network distillation* it's a way to generate curiosity rewards. If you want to know more about that, we wrote an article explaining this technique: https://medium.com/data-from-the-trenches/curiosity-driven-learning-through-random-network-distillation-488ffd8e5938

For this training, we'll modify one thing:

- The total training steps hyperparameter is too high since we can hit the benchmark (mean reward = 1.75) in only 1M training steps. To do that, we go to config/ppo/PyramidsRND.yaml,**and change max_steps to 1000000.**

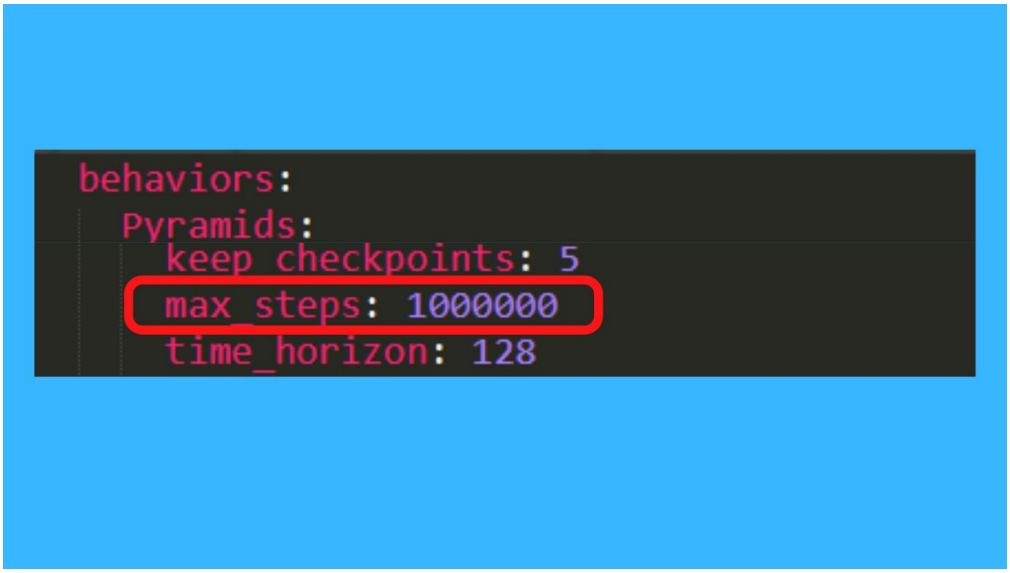


Figure 39: PyramidsRND configuration and maximum training steps [original figure]

As an experiment, you should also try to modify some other hyperparameters. Unity provides very good documentation explaining each of them here.

We're now ready to train our agent .

Train the agent The training will take 30 to 45min depending on your machine, go take a coffee you deserve it .

```
mlagents-learn ./config/ppo/PyramidsRND.yaml --env=./training-envs-executables/linux/Pyramids/
    Pyramids --run-id="Pyramids Training" --no-graphics
```

Push the agent to the Hugging Face Hub

- Now that we trained our agent, we're **ready to push it to the Hub to be able to visualize it playing on your browser.**

```
mlagents-push-to-hf  --run-id= # Add your run id  --local-dir= # Your local dir  --repo-id= # Your
    repo id  --commit-message= # Your commit message
```

Watch your agent playing https://huggingface.co/spaces/unity/ML-Agents-Pyramids

Bonus: Why not train on another environment? Now that you know how to train an agent using MLAgents, **why not try another environment?**

MLAgents provides 17 different environments and we're building some custom ones. The best way to learn is to try things on your own, have fun.

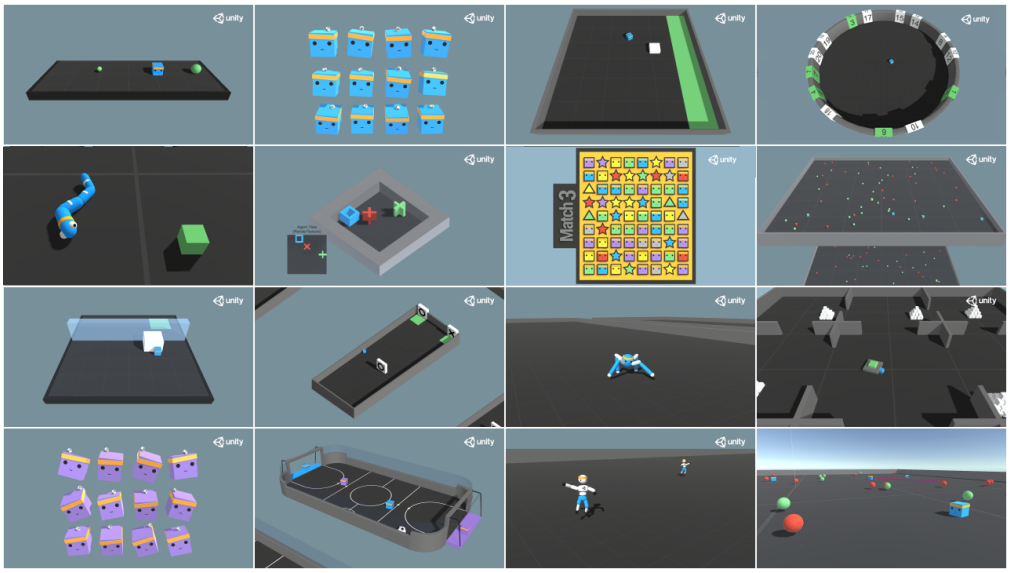


Figure 40: Example of another Unity ML-Agents environment [original figure]

You have the full list of the one currently available environments on Hugging Face here https://github.com/huggingface/ml-agents#the-environments

For the demos to visualize your agent https://huggingface.co/unity

For now we have integrated:

- Worm demo where you teach a **worm to crawl**.
- Walker demo where you teach an agent **to walk towards a goal**.

That's all for today. Congrats on finishing this tutorial!

The best way to learn is to practice and try stuff. Why not try another environment? ML-Agents has 18 different environments, but you can also create your own. Check the documentation and have fun!

See you on Unit 6 ,

Keep Learning, Stay awesome

6 Quiz

The best way to learn and to avoid the illusion of competence **is to test yourself.** This will help you to find **where you need to reinforce your knowledge**.

Q1: Which of the following tools are specifically designed for video games development?

- **A.** Unity (C#)
- **B.** Unreal Engine (C++)
- **C.** Godot (GDScript, C++, C#)
- **D.** JetBrains' Rider
- **E.** JetBrains' CLion
- **F.** Microsoft Visual Studio and Visual Studio Code

Q2: What of the following statements are true about Unity ML-Agents?

- **A.** Unity 'Scene' objects can be used to create learning environments
- **B.** Unit ML-Agents allows you to create and train your agents using Reinforcement Learning
- **C.** Its `Communicator` component manages the communication between Unity's C# Environments/Agents and a Python back-end
- **D.** The training process uses Reinforcement Learning algorithms, implemented in Pytorch
- **E.** Unity ML-Agents only support Proximal Policy Optimization (PPO)
- **F.** It includes a Gym Wrapper and a multi-agent version of it called `PettingZoo`

Q3: Fill the missing letters

- In Unity ML-Agents, the Policy of an Agent is called a b _ _ _ n
- The component in charge of orchestrating the agents is called the _ c _ _ _ m _

Q4: Define with your own words what is a `raycast`

Q5: Which are the differences between capturing the environment using `frames` or `raycasts`?

- **A.** By using `frames`, the environment is defined by each of the pixels of the screen. By using `raycasts`, we only send a sample of those pixels.
- **B.** By using `raycasts`, the environment is defined by each of the pixels of the screen. By using `frames`, we spawn a (usually) line to check what objects it collides with
- **C.** By using `frames`, we collect all the pixels of the screen, which define the environment. By using `raycast`, we don't use pixels, we spawn (normally) lines and check their collisions

Q6: Name several environment and agent input variables used to train the agent in the Snowball or Pyramid environments

Congrats on finishing this Quiz , if you missed some elements, take time to read the chapter again to reinforce () your knowledge.

6.1 Quiz solutions and explanations

Q1 solution

Correct answer(s): A, B, C

A. Unity (C#)

B. Unreal Engine (C++)

C. Godot (GDScript, C++, C#)

D. JetBrains' Rider Explanation: Although useful for its support of C# for Unity, it's not a video games development IDE

E. JetBrains' CLion Explanation: Although useful for its support of C++ for Unreal Engine, it's not a video games development IDE

F. Microsoft Visual Studio and Visual Studio Code Explanation: Including support for both Unity and Unreal, they are generic IDEs, not video games oriented.

Q2 solution

Correct answer(s): A, B, C, D, F

A. Unity 'Scene' objects can be used to create learning environments

B. Unit ML-Agents allows you to create and train your agents using Reinforcement Learning

C. Its `Communicator` component manages the communication between Unity's C# Environments/Agents and a Python back-end

D. The training process uses Reinforcement Learning algorithms, implemented in Pytorch

E. Unity ML-Agents only support Proximal Policy Optimization (PPO) Explanation: No, Unity ML-Agents supports several families of algorithms, including Actor-Critic which is going to be explained in the next section

F. It includes a Gym Wrapper and a multi-agent version of it called `PettingZoo`

Q3 solution

- b r a i n
- a c a d e m y

Q4 solution

A raycast is (most of the times) a linear projection, as a `laser` which aims to detect collisions through objects.

Q5 solution

Correct answer(s): C

A. By using `frames`, the environment is defined by each of the pixels of the screen. By using `raycasts`, we only send a sample of those pixels. Explanation: `Raycasts` don't have anything to do with pixels. They are linear projections (lasers) that we spawn to look for collisions.

B. By using `raycasts`, the environment is defined by each of the pixels of the screen. By using `frames`, we spawn a (usually) line to check what objects it collides with Explanation: It's the other way around - `frames` collect pixels, `raycasts` check for collisions.

C. By using `frames`, we collect all the pixels of the screen, which define the environment. By using `raycast`, we don't use pixels, we spawn (normally) lines and check their collisions

Q6 solution

- Collisions of the raycasts spawned from the agent detecting blocks, (invisible) walls, stones, our target, switches, etc.
- Traditional inputs describing agent features, as its speed
- Boolean vars, as the switch (on/off) in Pyramids or the `can I shoot?` in the SnowballTarget.

7 Conclusion

Congrats on finishing this unit! You've just trained your first ML-Agents and shared it to the Hub .

The best way to learn is to **practice and try stuff**. Why not try another environment? ML-Agents has 18 different environments.

For instance:

- Worm, where you teach a worm to crawl.
- Walker, where you teach an agent to walk towards a goal.

Check the documentation to find out how to train them and to see the list of already integrated MLAgents environments on the Hub: https://github.com/huggingface/ml-agents#getting-started

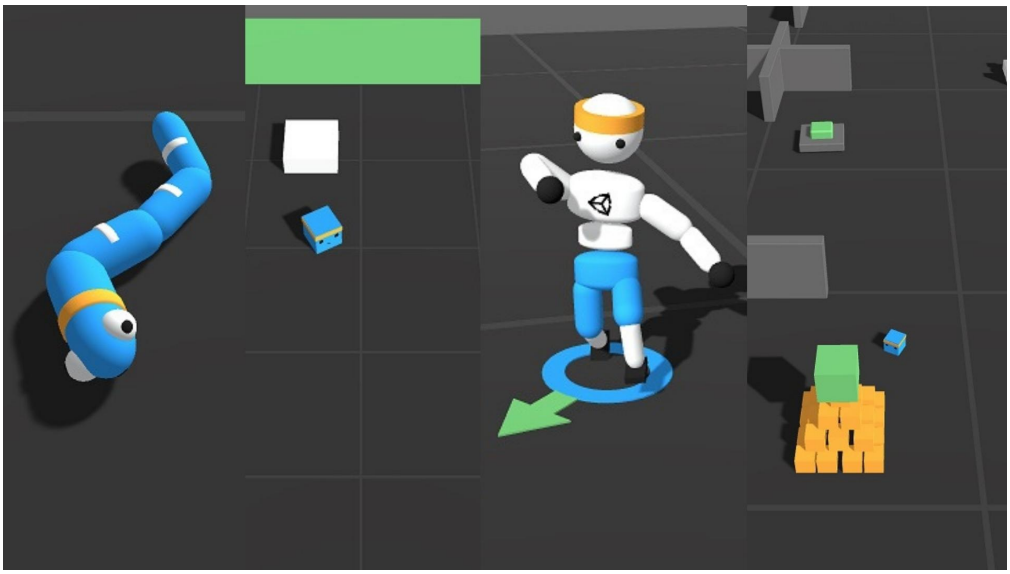


Figure 41: Additional example Unity ML-Agents environments [original figure]

In the next unit, we're going to learn about multi-agents. You're going to train your first multi-agents to compete in Soccer and Snowball fight against other classmate's agents.

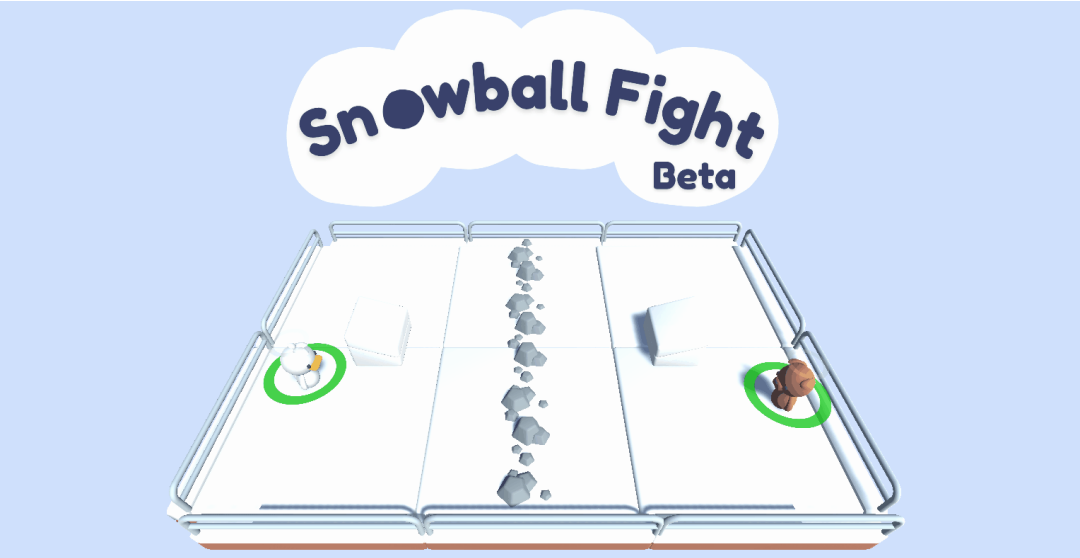


Figure 42: Snowball fight among trained agents (animation still) [original animation]

Finally, we would love **to hear what you think of the course and how we can improve it**. If you have some feedback then please fill this form

Keep Learning, stay awesome

8 Bonus: Learn to create your own environments with Unity and MLAgents

You can create your own reinforcement learning environments with Unity and MLAgents. Using a game engine such as Unity can be intimidating at first, but here are the steps you can take to learn smoothly.

8.1 Step 1: Know how to use Unity

- The best way to learn Unity is to do "Create with Code" course: it's a series of videos for beginners where **you will create 5 small games with Unity**.

8.2 Step 2: Create the simplest environment with this tutorial

- Then, when you know how to use Unity, you can create your first basic RL environment using this tutorial.

8.3 Step 3: Iterate and create nice environments

- Now that you've created your first simple environment you can iterate to more complex ones using the MLAgents documentation (especially Designing Agents and Agent part)
- In addition, you can take this free course "Create a hummingbird environment" by Adam Kelly

Have fun! And if you create custom environments don't hesitate to share them to the `#rl-i-made-this` discord channel.